

Lab 13 — Symbol Table Manager

2023-CS-18 M Ahmed

Compiler Construction Lab • May 22, 2026

1. Purpose

The goal of this lab submission is to provide a reusable symbol table module for the compiler construction project. The module supports one scope at a time through a *chained hash table*, and also supports a stack of scopes so that nested blocks and functions can be represented correctly. The implementation is written in Python and is intended to be reused by the recursive-descent parser from earlier labs.

2. Implemented Features

- Fixed-size table of **211 buckets**, matching lab requirements.
- **Separate chaining** for collision handling.
- Operations: `insert`, `lookup`, `lookup_current`, and `delete`.
- Scope management via `begin_scope` and `end_scope`.
- Proper **shadowing** of inner declarations over outer ones.
- Pretty-printing in aligned tabular format.
- Error reporting for **duplicate** and **undeclared** identifiers.

3. Task-by-Task Summary

3.1 Task 1 — Basic Hash Table

The module stores entries in an array of 211 slots. The hash function follows the **djb2** pattern from the lab handout, and each bucket uses linked-list chaining. Insertions occur in the current scope; lookups search upward through parent scopes; deletions remove names from the current scope only.

3.2 Task 2 — Scope Management

Each scope stores a *level number* and a *parent pointer*. Testing confirms correct nested lookup behaviour using global, function, and inner-block scopes.

3.3 Task 3 — Parser Integration

The parser should call the following API at the appropriate grammar points:

- `insert()` — on identifier declarations,
- `lookup()` — on identifier use,
- `begin_scope()` — when entering a block,
- `end_scope()` — when exiting a block.

3.4 Task 4 — Pretty Print

The `print_table()` method outputs aligned columns for: **ID**, **Name**, **Kind**, **Type**, **Scope**, and **Line**.

3.5 Task 5 — Test Report

Five test cases were prepared in the `test/` folder; corresponding outputs were saved in the `output/` folder.

4. Test Cases

Table 1 summarises all test results.

Table 1: Symbol Table Test Cases and Observed Results

#	Fixture File	Expected Result	Observed Result
1	<code>case01_basic_hash.txt</code>	Ten inserts succeed, five lookups execute, and three deletes remove names.	Final table contains seven names after deletion.
2	<code>case02_scope_shadowing.txt</code>	Three nested scopes created; inner <code>x</code> hides outer <code>x</code> .	Lookup resolves first to scope 0, then scope 2 after shadowing.
3	<code>case03_duplicates_undeclared.txt</code>	Duplicate declarations and undeclared names generate errors.	Output shows duplicate declaration and undeclared variable errors.
4	<code>case04_scope_dumps.txt</code>	Each scope prints on exit.	Scope 2, then 1, then 0 printed correctly.
5	<code>case05_pretty_print.txt</code>	Table output is aligned and readable.	Output shows proper headers and one row per symbol.

5. Observed Console Output

```
[Scope 0] Enter
[Scope 1] Enter
[Scope 2] Enter
[Scope 2] lookup x -> found at scope 0
[Scope 2] lookup y -> found at scope 0
[Scope 2] lookup x -> found at scope 2 after shadowing
[Scope 2] Exit, dump:
ERROR line 11: duplicate declaration 'x'
ERROR line 12: undeclared variable 'a'
[Scope 0] Exit, dump:
```

6. Limitations and Notes

The parser is *not* directly integrated in this submission. Instead, a separate test driver demonstrates the required symbol table functionality. The same module can later be imported into the parser without modifying the public API.

7. Conclusion

The implementation satisfies all required lab objectives:

- Hash-based insertion and lookup (djb2, 211 buckets, separate chaining).
- Nested scope handling with a parent-pointer stack.
- Correct shadowing of inner over outer declarations.
- Duplicate-declaration and undeclared-identifier detection.
- Formatted scope dumps on `end_scope`.